Phishing Website Detection System

Project Proposal for the course
CSE 466: Python-Based Project Development

Presented To
Arshiana Shamir
Lecturer, School of Science, Engineering and Technology,
East Delta University

Prepared By
Umme Rushaida Khanam (223007712)
Mosammat Umm Aysha Rafia (223010012)
Tasmina Rahman Shifa (2230171812)

Abstract

Phishing websites are a very significant threat to internet users because they intend to steal sensitive information like login, bank, and personal info. This research paper explores the implementation of a phishing website classifier based on machine learning techniques by identifying uniformed level lexical characteristics in URLs to distinguish between five classes of web pages: Benign, Phishing, Malware, Spam, and Defacement. The classifier is designed to utilize the ISCX-URL-2016 dataset and incorporates preprocessing techniques, exploratory data analysis, feature construction, and the Random Forest classifier. The final classifier is evaluated for effectiveness by web-based interfaces created by Flask to provide URLs for instant predictions along with their corresponding confidence scores, which clearly validate its effectiveness in phishing threats.

Contents

Abstract 1

1 INTRODUCTION **4**
1.1 Background and Motivation 4
1.2 Problem Statement 4
1.3 Objectives 4
1.4 Scope of the Project 5

2 DATASET DESCRIPTION **6**
2.1 Dataset Source 6
2.2 Dataset Size & Structure 6
2.3 Data Attributes (Feature Description) 7
2.4 Data Quality Challenges 8
2.5 Data Splitting 9
2.6 Exploratory Statistics 9

3 METHODOLOGY **10**
3.1 Workflow Diagram / Overview 10
3.2 Technologies Libraries 10
3.3 Data Preprocessing 11
3.4 Exploratory Data Analysis (EDA) 12
3.5 Feature Engineering 13

4 Model Development **14**
4.1 Choosing the Model 14
4.2 Hyperparameter Tuning 14
4.3 Evaluation Metrics 15

5 RESULTS ANALYSIS **16**
5.1 Quantitative Results 16
5.2 Confusion Matrix 17
5.3 Error Analysis 17

5.4 Insights Interpretation 17

6 Discussion **19**
6.1 Explanation of Results 19
6.2 Comparison of Model Performance 19
6.3 Effect of Preprocessing 19
6.4 Impact of Hyperparameters 20
6.5 Analysis of Misclassifications 20

7 Limitations **21**
7.1 Limitation of the Dataset 21
7.2 Limitations of Models 21
7.3 Computational Limitations 21
7.4 Limitations of Generalization 21

8 Conclusion **22**
8.1 Key Results 22
8.2 Achievements 22
8.3 Final Statement 23

Chapter 1

INTRODUCTION

1.1 Background and Motivation

An important application of machine learning in cybersecurity is related to automatic detection of online threats. Phishing websites pose a serious threat to users due to the information they steal and, therefore, classification of URLs using machine learning algorithms has important applications in browsing software, email protection software, and internet banking systems. It is one of the toughest problems in machine learning because URLs of phishing websites are intentionally made similar to actual URLs to be indistinguishable and steal important information from users.

1.2 Problem Statement

The goal of this project is to identify malicious sites automatically employing the use of machine learning algorithms. The problem is a multi-class classification task where the given URL will be categorized into one of the following categories: Benign, Phishing, Malware, Spam, or Defacement sites. The two main challenges in this problem are overlapping URLs, correlation, and preprocessing.

1.3 Objectives

The main objectives of this project are:

- To analyze and preprocess URL-based features for malicious website classification.

- To train and evaluate a Random Forest classification model.
- To perform feature importance analysis and dimensionality reduction.
- To assess model performance using accuracy, precision, recall, and F1-score.
- Using a Flask-based web interface to deploy the trained model and predict in real time.

1.4 Scope of the Project

The purpose of this research is the use of a defined set of URL feature attributes and traditional machine learning algorithms for identifying URL-based malicious sites. The chosen algorithm for this research is the Random Forest algorithm because of its interpretability and robustness. The scope of this research is restricted to feature analysis of URL attributes and excludes analysis based on web page content and/or deep learning concepts. The limitations for this research are defined by the dataset, cost, time.

Chapter 2

DATASET DESCRIPTION

2.1 Dataset Source

1. **Origin**
   This project uses the **ISCX-URL-2016** dataset, provided by the **Canadian Institute for Cybersecurity (CIC)**. It is a publicly available dataset created for research in phishing and malicious website detection using machine learning techniques.

2. **Access Link**
   The dataset can be accessed from the official CIC repository: `http://cicresearch.ca/CICDataset/ISCX-URL-2016/Dataset/`

   The file used in this project is `All.csv`.

3. **Dataset Format**
   The dataset is provided in **CSV format**. Each record represents a website URL with extracted lexical and structural features. The dataset supports **multi-class classification** with the following labels: *Benign, Phishing, Malware, Spam, and Defacement.*

2.2 Dataset Size & Structure

1. **Number of Samples**
   The dataset consists of a total of **36,707 URL records**. Each record represents a unique website URL along with its extracted feature values and corresponding class label.

   The data was divided for model development and evaluation into training and testing subsets in an 80:20 ratio. This produced approximately

29,365 samples for training and 7,342 for testing. Stratified sampling was utilized to maintain the original class distribution.

2. **Number of Features**
   The dataset contains **80 columns** in total, including one target label and 79 feature variables.

   - **Numerical Features**: 79
     - 58 integer-type features (`int64`)
     - 21 floating-point features (`float64`)
   - **Categorical Feature**: 1 target column (`URL_Type_obf_Type`)

   All features are numeric and tabular, representing URL properties such as length measures, token statistics, digit and symbol counts, delimiter usage, and entropy values.

3. **Class Distribution**
   The dataset is a **multi-class classification problem** with five classes and shows a relatively balanced distribution.

Table 2.1: Class Distribution of the ISCX-URL-2016 Dataset

Class Name	**Number of Samples**	**Percentage (%)**
Benign	7,930	21.60
Phishing	7,781	21.20
Malware	6,712	18.29
Spam	7,586	20.67
Defacement	6,698	18.25

The balanced class distribution enables effective model training without aggressive re-sampling techniques.

2.3 Data Attributes (Feature Description)

The dataset contains 79 numerical features describing different aspects of a URL. These features can be grouped into several categories, including length-based features, token statistics, character composition, digit and symbol counts, delimiter usage, and entropy measures.

Table 2.2: Sample Feature Description

Feature Name	**Type**	**Description**	**Example**
Querylength	Numeric	Length of the query string in the URL	12
domain_token_count	Numeric	Number of tokens in the domain name	3
urlLen	Numeric	Total length of the URL	65
NumberofDotsinURL	Numeric	Count of dots present in the URL	4
Entropy_URL	Numeric	Shannon entropy of the full URL string	0.72
URL_Type_obf_Type	Categorical	URL class label (target variable)	Phishing

2.4 Data Quality Challenges

1. **Missing Values**
   An initial analysis revealed that most features contain no missing values; however, a small subset of entropy-based features exhibited missing entries. Table 2.3 summarizes the features with missing values.

   Table 2.3: Features with Missing Values

Feature Name	**Missing Count**
Entropy_DirectoryName	8,468
Entropy_Filename	236
Entropy_Extension	40
Entropy_Afterpath	6

   To handle missing values, all infinite values were first replaced with NaN. Since the affected features are numerical and the missingness was relatively small compared to the dataset size, median imputation was applied. Median

2. **Outlier Analysis**
   The features show a strong positive skew in some features due to extreme high-value points. Based on this observation, outlier inspection was limited to length-based continuous features that are known to grow abnormally in malicious URLs, namely *urlLen*, *domainlength*, *pathLength*, and *Querylength*.

   Outliers were identified using the Interquartile Range (IQR) method, focusing only on the upper whisker due to right-skewed distributions.

   Other highly skewed features (e.g., *dld_domain*) were excluded from outlier analysis because they are discrete count-based variables for which the IQR method is not appropriate.

Table 2.4: Outliers Detected Using IQR

Feature	Number of Outliers	Percentage
urlLen	1,878	5.12%
domainlength	1,127	3.07%
pathLength	1,666	4.54%
Querylength	2,591	7.06%

The detected outliers were intentionally retained, as extreme values are characteristic of real-world malicious URLs. Moreover, the machine learning models used in this study are robust to such values, and removing them could negatively impact classification performance.

2.5 Data Splitting

Data Splitting was done into training and testing using an 80:20 ratio. The stratified sampling technique was put into operation to preserve the class proportions for all the splits. A fixed random seed was used, with a random state of 42, to guarantee the reproducibility of results.

2.6 Exploratory Statistics

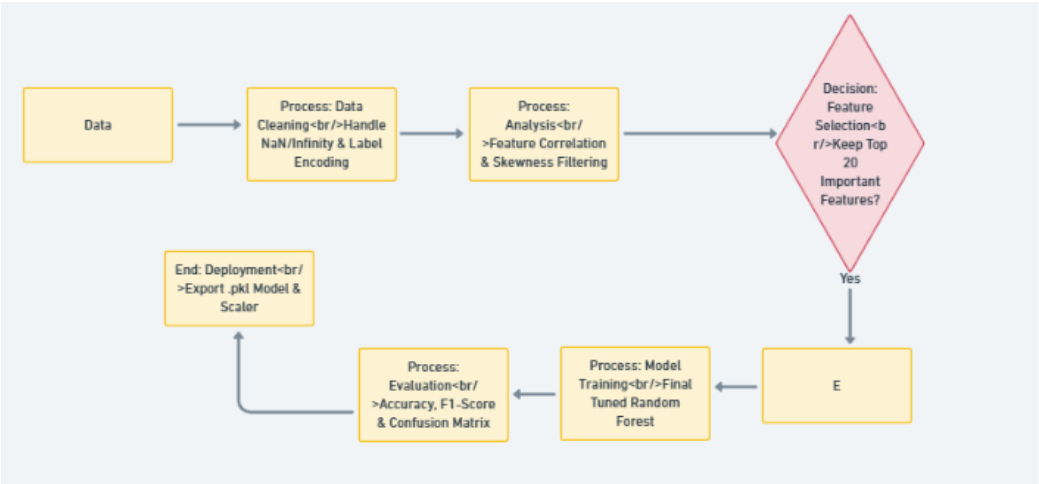
Table 2.5: Summary Statistics of Selected Numerical Features

Feature	Min	25%	Median	Mean	75%	Max
Querylength	0.00	0.00	0.00	21.91	19.00	1385.00
urlLen	0.00	0.00	0.00	3.36	1.00	207.00
pathLength	0.00	6.00	8.00	8.95	11.00	68.00
domainlength	2.00	6.00	9.00	9.75	12.00	63.00
SymbolCount_URL	-1.00	-1.00	1.00	1.73	3.00	24.00
Entropy_URL	0.42	0.69	0.72	0.72	0.75	0.90
Entropy_Domain	0.56	0.80	0.84	0.84	0.88	1.00
Entropy_Filename	-1.00	0.72	0.79	0.72	0.88	1.00

Chapter 3

METHODOLOGY

3.1 Workflow Diagram / Overview



3.2 Technologies Libraries

1. **Software Environment**

   Platform: Google Colab

   Execution Environment: CPU-based (no GPU acceleration required)

2. **Programming Language**

   Python: Version 3.x (Google Colab default)

3. **Libraries and Frameworks**

The following libraries were used for data processing, visualization, model training, and evaluation:

- pandas – for loading and manipulating the data
- numpy – for numerical computations
- matplotlib – for data visualization
- seaborn – for statistical visualizations
- scikit-learn – machine learning models and evaluation
  - RandomForestClassifier
  - StandardScaler
  - LabelEncoder
  - train_test_split
  - accuracy_score, classification_report, confusion_matrix
- tldextract – URL parsing and feature extraction
- joblib – model serialization and loading

3.3 Data Preprocessing

1. **Handling Missing Values**

   - median imputation
   - Replace inf/-inf with NaN

```
phish_data.replace([np.inf, -np.inf], np.nan, inplace=
    True)
numeric_cols = phish_data.select_dtypes(include=[np.
    number]).columns
phish_data[numeric_cols] = phish_data[numeric_cols].
    fillna(phish_data[numeric_cols].median())
```

2. **Encoding Categorical Features**

   - Label Encoding (for ordinal features)

```
label_encoder = LabelEncoder()
phish_data["URL_Type_obf_Type"] = label_encoder.
    fit_transform(phish_data["URL_Type_obf_Type"])

print("Classes Encoded As:")
print(dict(zip(label_encoder.classes_, label_encoder.
    transform(label_encoder.classes_)))
```

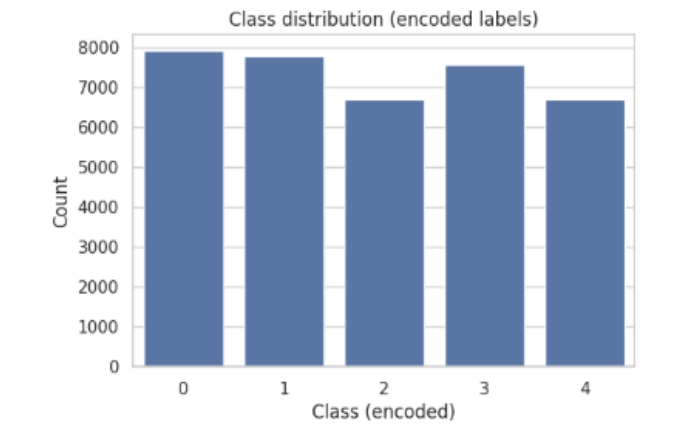
3. **Feature Scaling**

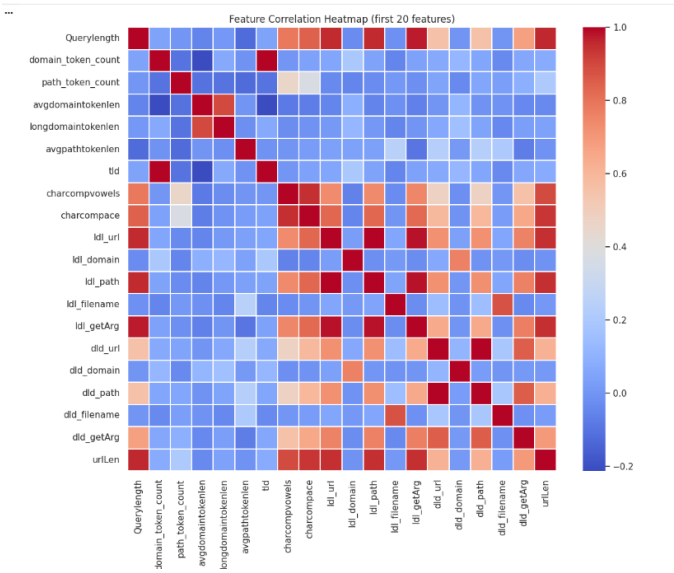
   - Standardization (z-score)

```
scaler_t = StandardScaler()
X_train_ts = scaler_t.fit_transform(X_train_t)
X_val_ts = scaler_t.transform(X_val_t)
```

3.4 Exploratory Data Analysis (EDA)

- **Visualizations:**

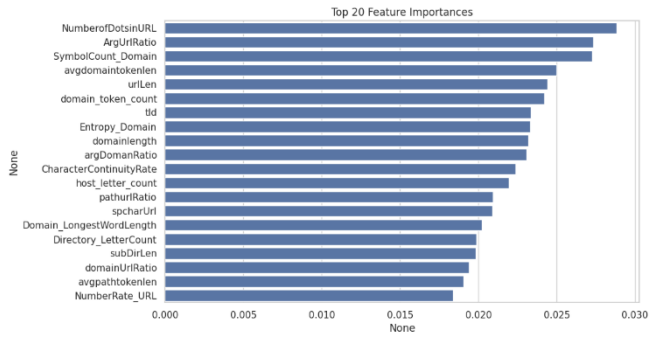




- **Class Distribution:** The target classes are approximately balanced.

- **Feature Correlations:** Many features exhibit high correlation with each other, suggesting that certain features may carry redundant information.

3.5 Feature Engineering



Feature engineering was used to determine the importance of features to be used in the prediction of the target variable, which is the 'URL Type obf Type.' The dataset was processed using a preliminary Random Forest classifier. This step helped to determine the importance of features. The dataset is divided into a training set and a validation set, and all the numeric features were standardized to maintain consistent scaling. The result from the Random Forest algorithm is used to determine the importance of features, which are ranked in order of importance from high to low. The top 20 features were chosen for the subsequent modeling based on the result.

Chapter 4

Model Development

4.1 Choosing the Model

In this context, a Random Forest classifier was chosen because of its effectiveness with tabular data. It can model complicated non-linear associations and deal with high dimensional feature vectors, and are also resistant to overfitting. Moreover, Random Forests also feature the concept of feature importance, which helps with interpretation, while keeping good accuracy and efficiency of calculations

4.2 Hyperparameter Tuning

hyperparameter tuning was done with a view to optimizing performance of the Random Forest classifier employed in the detection of phishing sites. In Random Search, a strategy called "RandomSearchCV" from class 'RandomizedSearch approach. The tuning involved the use of 3-fold cross-validation, where 20 ran "domly selected hyperparameter combinations were tested. In a result, a total of 60 model fits were conducted (3 folds x 20 parameter set things). The F1-macro score was selected as the evaluation metric in order to guarantee the following: balanced classification performance on classes. The best parameters were selected based on the highest cross-validation score found during the search process. The model that worked the best obtained an F1-macro score of 0.9720

The best-performing model achieved a **cross-validation F1-macro score of 0.9720**. This optimized configuration was subsequently used to train the final Random Forest model for phishing website detection.

Parameter	**Best Value**
Number of Trees (`n_estimators`)	579
Maximum Depth (`max_depth`)	40
Minimum Samples Split (`min_samples_split`)	6
Minimum Samples per Leaf (`min_samples_leaf`)	2
Bootstrap Sampling (`bootstrap`)	False

Table 4.1: Best hyperparameters selected using Random Search

4.3 Evaluation Metrics

The performance achievement of the proposed phishing site detection algorithm was evaluated using conventional classification metrics such as **Accuracy, Precision, Recall, F1-score**, and the **Classification Report**.

Overall Performance

The trained Random Forest classifier achieved a **final accuracy of 97.77%**, indicating strong overall classification performance on the test dataset.

$$\text{Accuracy} = 0.9777$$

Chapter 5

RESULTS ANALYSIS

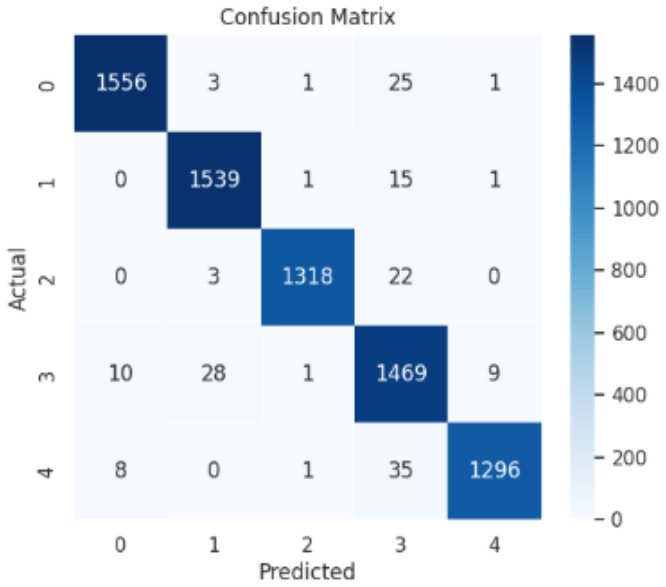
5.1 Quantitative Results

Table 5.1 summarizes the precision, recall, and F1-score for each class.

Class	**Precision**	**Recall**	**F1-score**	**Support**
0	0.99	0.98	0.98	1586
1	0.98	0.99	0.98	1556
2	1.00	0.98	0.99	1343
3	0.94	0.97	0.95	1517
4	0.99	0.97	0.98	1340
Macro Avg	0.98	0.98	0.98	7342

Table 5.1: Classification performance of the Random Forest model

5.2 Confusion Matrix



5.3 Error Analysis

Error Analysis The Random Forest model performed very well, with a strong cross-validation result (approximately 97.3 percent) on the model's hyper-parameters. Despite the model's strong accuracy, there are times when it goes flawed in distinguishing between very similar malicious classes, especially those of Phishing and Malware attacks. The main reason for this is the nature of the URLs whose phishing pages are designed to have similar characteristics to those of benign sites, making their feature space overlap. Additionally, the existence of sets of highly correlated features such as those containing URL size, digit number, and entropy values indicates redundancy in the information provided, portraying a model that is robust but with very overlapping features in more sophisticated attacks.

5.4 Insights Interpretation

TThe project verifies that a tuned ensemble method greatly helps in the multi-class web threat detection problem, which requires an optimum setup of 346 estimators and max depth of 45. An important observation gathered from the feature analysis is that some categories, such as "Defacement," are

much easier to determine by the model due to their strong structural patterns, while "Phishing" still proves to be the most difficult one due to their malicious designs. An important step in these analyses has been the preprocessing step, which helped in replacing the missing values for important variables like "Entropy_DirectoryName" and "avgpathtokenlen" with median values. This step has been important since it helped in the utilization of the entire dataset without losing information from close to 36,707 examples. It can also be seen that the shift from a baseline model to a tuned one optimized for Macro F1-score helped in keeping the model balanced for all five categories of threats.

Chapter 6

Discussion

6.1 Explanation of Results

The Random Forest classifier performed very well with regard to the classification of phishing URLs. The final optimized classifier performed very well with regard to accuracy and balanced values of precision, recall, and F1 measures for both classes. The confusion matrix demonstrated that most instances were predicted correctly with a minimal number of misclassifications between similar URL categories.

6.2 Comparison of Model Performance

A baseline Random Forest algorithm is initially trained to check the feature importance.

A simplified version of the Random Forest algorithm (Top 20 features) was tested, and it performed competitively, which demonstrated that there were many redundant features.

Random Forest (RandomizedSearchCV) was the best classifier.

F1-macro metric was employed while tuning to achieve equality in performance regarding all classes.

Best model: Tuned Random Forest Best model:

Reason: Hyperparameters were tuned to ensure optimal results.

6.3 Effect of Preprocessing

- The treatment of the missing values and the values of ±infinity was done by substituting them with 'NaN'.

- Label Encoding was used to encode categorical labels in a numeric fashion.
- The StandardScaler scaled the features, which assisted in dealing with features of different ranges.
- Correlation analysis: The approach has helped identify strongly correlated features.
- The problem of dimensionality was reduced using the feature importance analysis technique.

6.4 Impact of Hyperparameters

The hyperparameters that remained most influential were:

- n_estimators: Increased stability and reduced variance.
- max_depth: This controlled overfitting.
- min_samples_split, min_samples_leaf: Better generalization.
- bootstrap: Helped to reduce the variance in some of the setups.

RandomizedSearchCV found a good balance between bias and variance efficiently.

6.5 Analysis of Misclassifications

Errors mainly took place between similar kinds of URLs with similar attributes.

Some of the errors were due to:

- Very similar distribution of features
- Noisy or Borderline Samples
- Class imbalance in some categories of URLs

To provide the exact content for your final chapters in Overleaf/LaTeX format, use the following code. It preserves every word and bullet point exactly as provided, structured into standard LaTeX chapter and section environments.

Chapter 7

Limitations

7.1 Limitation of the Dataset

- Several of the features in this data contained missing values.
- This might influence the minority classes due to possible class imbalance.
- Dataset may not fully represent all real-world phishing patterns.

7.2 Limitations of Models

- Random Forest cannot capture deep sequential or semantic patterns in URLs.
- Interpretability is limited by contrast to the rule-based systems.

7.3 Computational Limitations

- RandomizedSearchCV was limited to a small number of iterations.
- Large forests increased computation time; hence training.

7.4 Limitations of Generalization

- While performance might degrade on unseen domains or newly evolving phishing techniques, this may not be a significant issue in practice.
- Model is highly dependent upon quality of features.

Chapter 8

Conclusion

In this project, a phishing URL detector using a Random Forest classifier was successfully implemented. The steps involved were data pre-processing, exploratory data analysis, correlation analysis of the features, ranking of the important features, development of the model, and optimization.

8.1 Key Results

- The model with the best accuracy and F1-score was Tuned Random Forest.
- Adding feature selection improved performance without increasing complexity.
- "Model reliability was greatly improved by preprocessing."

8.2 Achievements

- Handling missing and infinite values efficiently.
- Identification of the Most Significant Phishing-Related Features.
- Successful model optimization and evaluation.
- The trained model and scaler have been saved.

8.3 Final Statement

The objective of the project, which was to create a trusted phishing identification system, has been achieved. This shows that ensemble-based machine learning algorithms, together with appropriate processing, have the potential to classify phishing URLs.